

FastAPI + SQLAlchemy

Study notes on the *SQL (Relational) Databases* tutorial · quiz score 6/10 · 9 Oct 2026

Organized by topic. Lines marked **Your mistake** are things you got wrong on the quiz. Every section ends with one line to remember.

1. The big picture

Your Python code never writes SQL directly. SQLAlchemy sits in the middle and translates both ways: classes and objects become SQL, and the rows the database returns become Python objects again.

```
Client  --JSON--> FastAPI  --objects--> SQLAlchemy  --SQL--> Database
        <-JSON---          <-objects---          <-rows---
```

SQLAlchemy = **SQLAlchemy** (talks to the database) + **Pydantic** (validation and JSON). A class with `table=True` is a real table. Without it, it's just a data shape. Add `echo=True` to `create_engine` to print every SQL statement it runs.

Engine: holds the connections. One per app. Created when Python imports your file, destroyed when the process ends (Ctrl+C). In dev mode, every save reloads the app and makes a new one.

Session: one unit of work. One per request, opened and closed by `get_session()`. The code after `yield` runs after the response is sent.

Your mistake: you said the engine stops when there's an error. A failed request only affects its own session.

→ **Engine lives as long as the app. Session lives as long as one request.**

2. Running queries: what you actually get back

`session.exec(query)` does **not** return a `Hero`. It returns a **result object** (`ScalarResult`): an unopened box of rows. You then choose how to open it.

Why two steps? To SQL, a query can match **0, 1, or many rows**, even a `WHERE id = 5`. Only you know what you expect, so you say it: one row, the first row, or all rows. The box also lets you stream rows without loading everything into memory.

Code	Type you get	If nothing matches
<code>session.exec(select(Hero))</code>	<code>ScalarResult[Hero]</code> (the box)	empty box, still truthy
<code>... .first()</code>	<code>Hero</code> <code>None</code>	<code>None</code>
<code>... .all()</code>	<code>list[Hero]</code>	<code>[]</code>
<code>... .one()</code>	<code>Hero</code>	raises an error (also if more than one)
<code>session.get(Hero, 5)</code>	<code>Hero</code> <code>None</code>	<code>None</code>

get vs select. `session.get` only looks up by **primary key** and is the shortcut for that. It may even skip the database if the session already has that object. Use `select(...).where(...)` for

anything else: filter by name, age range, sorting, pagination.

Use **.first()** when you expect at most one row. Use **.all()** when you expect a list.

Your mistake: you said `exec` returned a `Hero` and the 404 would fire. The box is truthy even when empty, so `if not hero` never fires; FastAPI then fails to turn the box into a `Hero` → **500**, not 404.

→ `exec` gives a box → open it with **.first()** or **.all()**. **get(Model, id)** gives the object.

3. Checking whether a record exists

Condition	True when hero is...
<code>if not hero:</code>	anything falsy: <code>None</code> , <code>0</code> , <code>""</code> , <code>[]</code> , <code>{}</code>
<code>if hero is None:</code>	exactly <code>None</code> , nothing else
<code>if hero is not None:</code>	anything except <code>None</code> , including <code>0</code> and <code>""</code>

For a `Hero` object both `not hero` and `hero is None` work, because model objects are always truthy. Still prefer `is None`: it says exactly what you mean, and it stays correct when a query returns a plain value like a count of `0` or an empty string, which `not` would wrongly treat as "missing".

```
hero = session.get(Hero, hero_id)
if hero is None:      # not found
    raise HTTPException(status_code=404, detail="Hero not found")
```

→ **Checking for 'not found'? Write `is None`.**

4. When things go wrong: errors become 500

This is your biggest gap. **Pydantic checks the shape** of the request: types and required fields. It never looks inside the database. **The database checks the rules**: unique keys, NOT NULL, existing columns. If the database rejects something and you don't catch it, FastAPI returns **500 Internal Server Error**.

What happened	Who catches it	Client gets
wrong type or missing field	Pydantic	422
you raised <code>HTTPException(404)</code>	your code	404
duplicate primary key or unique value	database → <code>IntegrityError</code>	500
<code>None</code> into a NOT NULL column	database → <code>IntegrityError</code>	500
column doesn't exist in the table	database → <code>OperationalError</code>	500

Your mistake: for a POST with an existing id you guessed 424. Validation passes (it's a valid int), the DB rejects the duplicate → `IntegrityError` → 500. 424 means "Failed Dependency" and is unrelated.

How to handle it. For id: don't accept it from the client at all. That's why HeroCreate has no id. For a field the client must send but that has to be unique (like email): catch IntegrityError around the commit, call session.rollback(), and return **409 Conflict**. Don't check first with get: two requests can both check, both see "free", and both insert.

→ **Pydantic checks shape, the DB checks rules, and any uncaught exception is a 500.**

5. Multiple models: one model per job

Version 1 used one class for everything, so the client could send id and secret_name leaked in every response. Version 2 splits it:

Model	Job	id	secret_name
HeroBase	shared fields (parent)	-	-
Hero	the table	int None	yes
HeroCreate	POST body	no	yes (accepted)
HeroUpdate	PATCH body, all optional	no	optional
HeroPublic	the response	int	no (never returned)

Hero.model_validate(hero_create) copies the input into a table object.

response_model=HeroPublic filters the output. It's used instead of -> HeroPublic because the function really returns a Hero, and the type checker would complain.

Why two different id types? Same field, different moments. A Hero exists *before* saving, when it has no id yet, so it must allow None. HeroPublic is only used *after* saving, when the id always exists, so id: int is a promise to clients that they never need to check for None.

Your mistake: you explained only the Hero half. The point of HeroPublic's type is the guarantee.

Passwords follow the secret_name pattern: accept them, never return them, and hash them before storing.

→ **Before saving, id may be None. After saving, it's guaranteed, so promise it.**

6. PATCH and exclude_unset

hero.model_dump() returns **every** field, filling unset ones with their defaults (None).

hero.model_dump(exclude_unset=True) returns **only the fields the client actually sent**. That's how you keep untouched fields from being overwritten.

Database row: name=Eyad, age=21, secret_name=esa.

PATCH body	With exclude_unset	Without exclude_unset
{}	nothing changes	tries to set everything to None → 500
{"age": null}	age becomes NULL (client asked for it)	also nulls name and secret_name → 500
{"name": "Ali"}	only name changes	secret_name=None breaks NOT NULL → 500, nothing saved

Your mistake: you said age and secret_name both become None. The dump does say that, but secret_name is NOT NULL in the database, so the commit fails, the transaction is rolled back, and age stays 21.

→ **exclude_unset** separates "the client didn't mention it" from "the client sent null".

7. Changing the schema after the table exists

Editing the class only changes Python. The table in database.db is a separate thing. create_all only creates tables that are **missing**. If hero exists, it skips it completely and never adds, removes, or alters columns.

So after adding email: str and restarting: no new column. The next POST runs an INSERT that includes email → OperationalError: table hero has no column named email → 500.

Your mistake: you got the first part, but missed the error on the next POST, and deleting the file is only OK in development. In production it wipes every row.

Doing it by hand from Python (fine for learning):

```
from sqlalchemy import text

with engine.begin() as conn:          # begin() commits automatically at the end
    conn.execute(text("ALTER TABLE hero ADD COLUMN email TEXT"))
```

Note: existing rows have no email, so adding a **NOT NULL** column needs a DEFAULT value, or SQLite refuses.

sqlite3 vs the engine. Python's built-in sqlite3 module only talks to SQLite. The SQLAlchemy engine is database-agnostic: the same code works on PostgreSQL by changing the URL, and it manages connections and transactions for you. Since your app already has an engine, use it.

Why migrations (Alembic) in production instead of manual ALTER TABLE? Each change becomes a versioned script stored in git. It runs the same way on your laptop, your teammates' machines, and the server. It can be undone with a downgrade. Alembic can also generate the script by comparing your models to the database. Manual commands are easy to forget, run twice, or run differently on each machine.

→ **create_all** is a dev convenience. **Production schema changes go through Alembic migrations.**

8. Defining the same table twice

```
InvalidRequestError: Table 'hero' is already defined for this MetaData instance.
Specify 'extend_existing=True' to redefine options and columns.
```

Every class with table=True registers its table in SQLAlchemy.metadata. Your file defines Hero twice (v1 and v2), so the second registration crashes on import and **the app never starts**. extend_existing=True only silences the error. The real problem is two definitions of one table.

Routes have a similar issue: with the same method and path registered twice, the first one wins and the second is dead code.

Your mistake: you said "the first one is used". That's true for routes, but the table error stops everything before any route runs.

→ **One table class per table. Keep the versions in separate files: v1.py and v2.py.**

9. Fix your file

Line	Problem	Fix
85	exec without .first() → 404 never fires	.first() or session.get
98	comment says "if hero is not None" (backwards)	if hero is None: # not found
7, 116	Hero defined twice → crash on import	split into v1.py and v2.py
74, 157	pagination without ORDER BY: page order not guaranteed	.order_by(Hero.id)
46	@app.on_event is the older style	fine for now; lifespan is current

10. Practice: break it on purpose

Run with echo=True, use /docs, and read the traceback each time:

1. POST a hero with an id that already exists → IntegrityError.
2. Remove exclude_unset, then PATCH only the name → NOT NULL failure.
3. Add email: str to Hero without deleting the db, then POST → OperationalError.
4. Run your original read_hero with a missing id → 500. Then fix it and get 404.
5. Stretch: add a unique email and return 409 on duplicates.

Review in this order

1) What exec returns · 2) result types · 3) get vs select · 4) None, truthiness and 404 · 5) errors → 500 · 6) exclude_unset · 7) schema changes and migrations · 8) duplicate tables.
The first four build on each other.